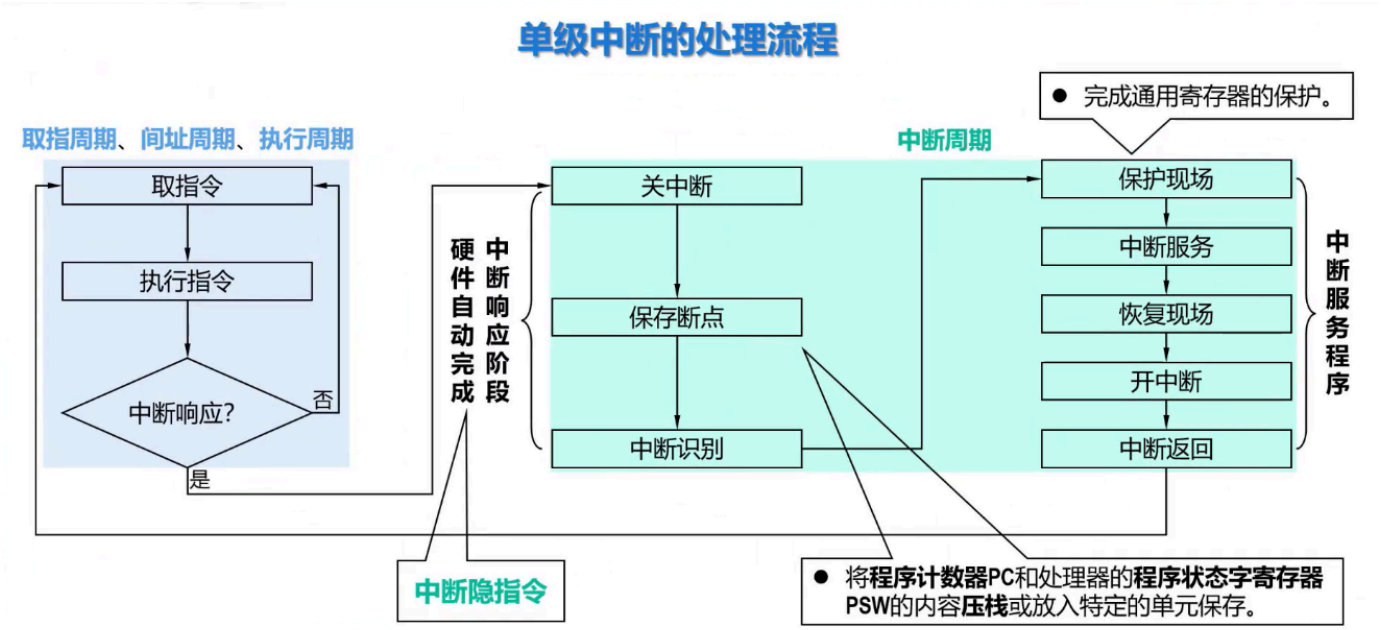
一、单机中断处理流程图



左侧蓝色区：正常执行循环与响应时机

CPU平时在"取指周期→（间址周期）→执行周期"的流水里循环。每条指令**执行完毕**，硬件检查菱形框"中断响应？"：

- **否** → 回到取指指令，继续下一条；
- **是** → 进入**中断周期**（绿色区）

两个必考细节：

1. **响应边界是"指令执行周期结束"**，CPU绝不会在指令执行到一半时被可屏蔽中断打断——中断请求最多等到这条指令执行完；
2. 响应需要同时满足的条件：**有中断请求 + CPU开中断（IF=1）+ 请求未被屏蔽**，缺一不可

这里还要建立一个重要概念：**中断周期是CPU的第四种工作周期**，与取指、间址、执行周期并列。也就是说"响应中断"在CPU眼里也是一段时间内的工作阶段，并非凭空插入

中间绿色区：中断周期 = 中断隐指令（全程硬件自动）

进入中断周期后，CPU硬件自动完成三个动作，图下方标注"中断隐指令"——它不是一条真实的指令，没有操作码、不出现在程序里、程序员不可见，纯粹是硬件在响应中断时的一套固定微操作序列

- **动作① 关中断**：清IF位。为什么放第一步？为了**保证后续"保存断点"的原子性**——如果断点压栈压到一半又被新中断打断，栈里的断点就废了，原程序再也回不去了。关中断就是给接下来的硬件操作上一把锁
- **动作② 保存断点**：按图中注释，把**PC和PSW一起压栈（或存入特定单元）**。PC是"回来后从哪继续取指"，PSW是"回来后恢复什么运行状态（含IF位本身）"。注意这是**硬件自动压栈**，不是软件做的
- **动作③ 中断识别**：判明是哪个中断源请求，并找到对应中断服务程序的入口地址——这一步对应你8259A流程图里的**第二个INTA脉冲**：8259A把中断类型号放上数据总线，CPU取回后查中断向量表，得到ISR入口地址送入PC。到此硬件任务全部完成，图中的箭头从"中断识别"指向右侧"保护现场"，控制权正式移交软件

==**隐指令只做这三件事，多一件都没有**——"保护现场"不属于中断隐指令，这是408最爱挖的分界线==

右侧绿色区：==中断服务程序（软件执行，五个步骤按序背）==

从"保护现场"开始，全是**程序员编写的ISR在跑**：

- **① 保护现场**：把**通用寄存器**内容压栈（图中气泡注明"完成通用寄存器的保护"）。注意与断点的分工：**断点（PC+PSW）硬件已存，现场（通用寄存器）软件来存**——因为中断服务程序要用这些寄存器，不保护就会破坏被中断

程序的运行环境。
- ② **中断服务**：干正事，比如读走键盘缓冲、启动打印机。**单级中断下这一段全程关着中断**，任何新请求都只能等。
- ③ **恢复现场**：把通用寄存器从栈里弹回。注意它必须发生在**开中断之前**——如果先开中断再恢复现场，恢复到一半被新中断打断，寄存器又被别人改了，现场就乱了。
- ④ **开中断**：置回IF位。**这是单级中断的标志性位置**——开中断只出现在服务程序末尾。为什么必须有这一步？因为隐指令阶段关了中断，如果一直不开，返回主程序后CPU将永远不再响应任何中断，系统就"聋"了。
- ⑤ **中断返回**：执行IRET，硬件自动弹出断点（PC和PSW），回到被中断的程序，图上的箭头绕回左侧"取指指令"，回到主循环。

把整条时间线串成一段话

> CPU正在执行主程序，每执行完一条指令就瞄一眼中断请求线。某次执行完，发现有请求且IF=1，于是进入中断周期：硬件先关中断锁住现场，再把PC和PSW压栈保住退路，然后通过中断识别找到服务程序入口；接着软件登场，先压栈保护通用寄存器，再专心处理中断请求（全程无打扰），处理完弹栈恢复寄存器，然后开中断恢复响应能力，最后IRET弹回PC和PSW，从断点处继续执行主程序

"单级"的题眼：与多级中断的对比

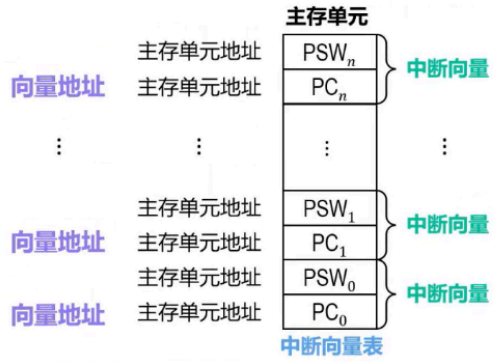
这张图必须和"多级中断"放在一起对比记忆，考点全在**开中断的位置**：

对比项	单级中断（本图）	多级中断
服务程序执行期间	**全程关中断**，不允许被打断	保护现场后**立即开中断**，允许更高优先级请求插入
开中断的位置	恢复现场之后、中断返回之前（末尾一次）	保护现场之后开一次（放行嵌套）；恢复现场前关一次；恢复完再开，然后返回（**出现两次**）
中断嵌套	不可能发生	可形成嵌套链
硬件代价	小	需要优先级判别逻辑（如8259A的PR）
记忆口诀：**单级“一关到底，最后才开”；多级“早开晚关，夹住现场”**——多级中“恢复现场前必须先关中断”，原理与单级中“恢复现场前不能开中断”完全一致。		
### 易错点清单		
1. **中断隐指令 vs 中断服务程序的分界**：隐指令=关中断、保存断点、中断识别（硬件）；保护现场及之后=软件。2010年真题就考“哪些操作在ISR内完成”，答案里绝不能混入隐指令的动作；		
2. **断点 vs 现场**：断点=PC+PSW（硬件压栈），现场=通用寄存器（软件压栈），两者保存的时机和执行者都不同；		
3. **保护现场不在隐指令里**：同理，**恢复断点也不由软件做**——IRET执行时硬件自动弹PC和PSW，ISR里不用手写；		
4. **关中断必须最先做**：它保护的是“保存断点”这个动作本身；		
5. **开中断的位置是单级/多级判断题的判据**：看到流程图中“开中断”出现在保护现场之后 → 多级；出现在恢复现场之后 → 单级；		
6. **中断响应不发生在指令中间**：选项说“CPU随时可响应中断”必错（NMI也只是在指令边界优先级更高地被响应，不是插在指令中间）；		
7. **中断识别≠软件查询才叫识别**：向量中断由硬件排队器+编码器直接给出向量地址（对应8259A送类型号），软件查询法才是服务程序里逐个查状态位。		
一句话收束：**指令边界响应 → 硬件三连（关中断/存断点/找入口）→ 软件五连（护现场/办正事/复现场/开中断/回家）**；单级中断的全部精髓，就是中间那段“办正事”时大门紧锁，直到收拾完行李才重新开门		

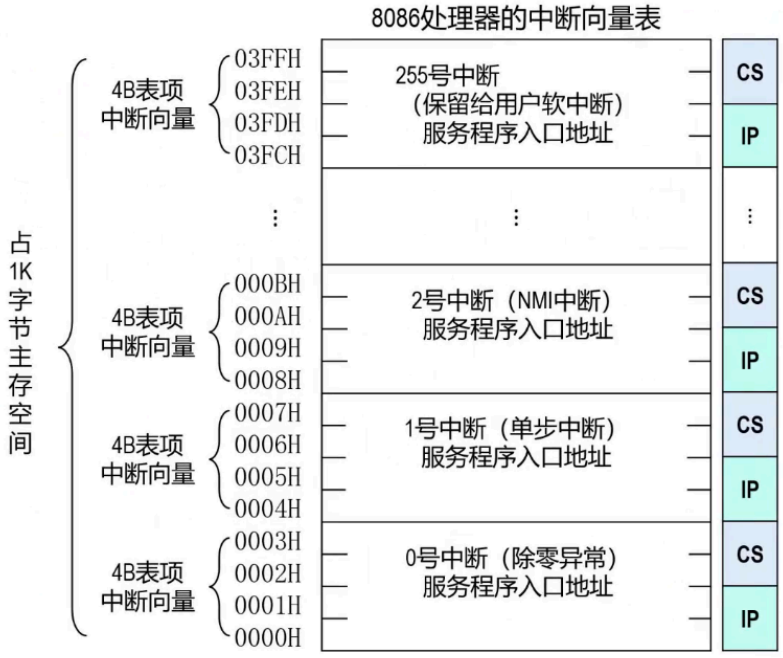
二、中断向量、中断向量表、向量地址

中断向量、中断向量表、向量地址

- CPU将**中断号**作为**索引**，在**中断向量表**中查找**中断服务程序的入口地址**，以便实现程序的转移。
- **中断向量**：一般将**中断服务程序的入口地址PC**和**程序状态字PSW**合称为中断向量。
  - 某些计算机的中断服务程序需要载入**PSW**才能运行。
  - 对于**不需要载入PSW**即可运行中断服务程序的计算机，中断向量就是**中断服务程序的入口地址PC**。
- **中断向量表**：由**若干中断向量**构成（中断向量的集合）。
  - 通常**存放在主存中**，在操作系统引导过程中初始化。
  - 利用**中断号**对其进行索引访问。
- **向量地址**：中断向量表中各表项的地址，也称为**中断指针**。



中断向量表的举例



- 将**中断号n**作为**索引**，对中断向量表中的中断向量（4B表项）进行访问。
  - **中断向量在中断向量表中的位置**：$n \times 4$
- 中断向量地址的高2个字节的内容，用于更新到**代码段寄存器CS**。
- 中断向量地址的低2个字节的内容，用于更新到**指令指针寄存器IP**。
- 8086处理器中，**代码段寄存器CS**和**指令指针寄存器IP**共同决定了CPU将要执行的下一条指令在**主存中的物理地址**。
  - **代码段寄存器CS**：存放当前代码段的起始地址，CPU在计算物理地址时，会将其左移4位。
  - **指令指针寄存器IP**：存放相对于当前代码起始地址的偏移地址。
  - **主存中的物理地址**：(CS) << 4 + (IP)